Clinical Decision Support System for Disease Risk Assessment

Midterm Report

Student Names:

Uthara Uday (300395271)

Akiful Mohaimin (300369823)

Course: **CSIS 4495 – Applied Research Project**

Section: **03**

Team Lead: **Akiful Mohaimin**

Video Presentation : Click here to see

Contents

1. Introduction 4
1.1 Background and Domain Context 4
1.2 Framing the Research Problem 4
1.3 Literature Review and Identified Research Gap 5
1.4 Hypotheses, Assumptions, and Expected Contributions 5
2. Summary of the Initially Proposed Research Project 6
3. Changes to the Proposal 7
3.1 Prioritization of Document Processing 7
3.2 Architectural Consolidation Using Streamlit 7
3.3 Phased Machine Learning Implementation 8
3.4 Refinement of Indicator Scope 8
4.1 Updated Timeline and Milestones 8
Weeks 7–8: Data Analytics and Feature Validation 9
Weeks 8–9: Machine Learning Model Development and Evaluation 9
Weeks 9–10: Integration of Analytics into Web Application 10
Weeks 10–11: System Testing and Data Validation 10
Weeks 11–12: System Refinement and Interpretability Enhancement 10
Weeks 12–13: Final Report and Presentation 10
4.2 Individual Responsibilities (From Midterm Onward) 11
Uthara Uday 11
Akiful Mohaimin 11
5. Implemented Features (Midterm Milestone) 11
5.1 Secure PDF Upload and Validation Module 11
Feature Description 11
Design Considerations 12
Screenshot of the interface in Streamlit 12
Individual Contribution 13
5.2 PDF Parsing and OCR Extraction Pipeline 13
Feature Description 13
Design Considerations 13
Implementation Details 14
Screenshot of Extracted text preview in UI 14
Individual Contribution 14

5.3 Preprocessing and Structured Indicator Extraction 15
Feature Description 15
Design Considerations 15
Implementation Details 15
Screenshot of JSON output 15
Individual Contribution 16
5.4 Integrated Streamlit Web Application 16
Feature Description 16
Design Considerations 17
Screenshot of the full application interface: 17
Individual Contribution 23
5.5 End-to-End Workflow Validation 23
6. AI Use Section 24
7. Work Date / Hours Log 24
Student Name: Uthara Uday 24
8. Closing 25
8.1 References 26
9. APPENDIX 26
Appendix A – Prompt History 26

1. Introduction

1.1 Background and Domain Context

The global healthcare systems have developed a trend of using digital means through which patients now have direct access to their personal medical data in the form of lab results, diagnostic summaries, and standardized health records. This change to patient-friendly digital records, as described in the initial project proposal, fosters transparency and participation. However, is not the only thing that ensures understanding.

Medical records that are facing the patient are often full of complicated clinical terms, numerical lab values, reference ranges, and unstructured narratives. The reports are mainly intended to be interpreted by the healthcare professionals, as opposed to non-clinical users. Thereupon, patients might not perceive the clinical importance of anomalous laboratory results or to identify the initial signs of possible risks of diseases.

Clinical Decision Support System (CDSS) use has long been designed to support the work of clinicians in their decision-making through the analysis of structured electronic health record (EHR) data. Shortliffe and Sepulveda (2018) state that efficient CDSS tools must offer valuable support to assist clinical judgment in a timely manner and be valuable and interpretable. In the same manner, Rajkomar et al. (2019) show that machine learning models are able to detect the patterns of diseases successfully when trained on structured healthcare datasets. Nevertheless, at the time of this study, little of this research has addressed patient-friendly, unstructured documents and has concentrated more on clinician-facing systems and institutional data.

The study falls in the new area of patient-centered healthcare analytics. It explores the way unstructured, patient-readable medical reports, specifically PDF lab reports, can be converted into structured and interpretable output by document processing and an explainable system design.

1.2 Framing the Research Problem

There is still a significant disconnect between the availability and interpretability of medical data, despite patients having previously unheard-of digital access to it. The primary research issue this study attempts to address is:

How can unstructured medical reports that are accessible to patients be methodically processed and converted into comprehensible, structured indicators that promote awareness of the risk of non-diagnostic diseases?

To solve this issue, the research questions that are studied in the paper are as follows:

1. How can clinically relevant information be extracted from diverse medical reports using PDF parsing and optical character recognition (OCR) techniques?
2. How can unstructured and noisy extracted text be transformed into structured features through cleaning and normalization?

3. How can extracted indicators be communicated to non-clinical users in an open, morally sound way?
4. How can a clinical decision support prototype that interacts with patients stay non-diagnostic while still offering insightful analysis?

These are crucial questions because misreading medical reports can result in needless worry, postponed clinical consultations, or dependence on dubious sources of information. A method that is organized and easy to understand could improve patient comprehension and promote knowledgeable interaction with medical professionals.

1.3 Literature Review and Identified Research Gap

Using structured EHR data, previous studies have shown that machine learning is effective in predicting disease risk (Rajkomar et al., 2019). Additionally, it has been determined that explainable artificial intelligence (XAI) is crucial in healthcare settings, especially when systems communicate with non-expert users (Holzinger et al., 2019). Transparency and avoiding automated diagnosis are necessary for the ethical framing of CDSS tools, as the proposal emphasizes.

The majority of research focuses on institutional analytics pipelines rather than patient-facing applications, even though the body of existing literature supports the technical viability of OCR and rule-based extraction from medical documents. Conventional predictive modeling frameworks often do not include unstructured patient-accessible documents, such as downloadable PDF lab reports. The key research gap identified in this project is the limited integration of:

- Unstructured document processing (PDF + OCR),
- Structured feature generation,
- Patient-centered interface design, and
- Explainable, non-diagnostic decision support.

By creating a functional prototype that transforms patient-accessible documents into structured indicators appropriate for upcoming risk modelling, this study helps close that gap.

1.4 Hypotheses, Assumptions, and Expected Contributions

This research is based on the following assumptions:

1. Enough textual and numerical indicators are included in patient-accessible laboratory reports to facilitate organized disease risk awareness.
2. Clinically significant values can be consistently extracted from a variety of report formats using OCR and rule-based extraction techniques.
3. Instead of relying on automated diagnostic results, patients look for informational support.
4. Without taking the place of expert medical advice, a clear and organized presentation of extracted indicators can enhance interpretability.

Based on these assumptions, the primary hypothesis of this study is:

Patients will be better able to comprehend possible health risks and interact with healthcare providers more actively if unstructured patient-accessible medical reports are converted into structured and understandable indicators.

Both academic and practical contributions are anticipated from this study. Academically, the project advances applied research in patient-centered system architecture, explainable AI, and healthcare data preprocessing. From a practical standpoint, it shows that it is possible to create a multimodal clinical decision support prototype that preserves ethical and non-diagnostic boundaries while bridging the gap between unprocessed medical data and patient understanding.

2. Summary of the Initially Proposed Research Project

According to the approved proposal, the original research project's goal was to create a patient-centered Clinical Decision Support System (CDSS) for evaluating disease risk. The main goal was to close the gap between medical records that patients could access and those that non-clinical users could meaningfully interpret. The project was presented as an applied research study that combined explainable artificial intelligence, machine learning, and healthcare analytics.

The proposed system was conceptualized as a multimodal decision support framework capable of processing heterogeneous healthcare data formats. These included:

- PDF laboratory reports and scanned medical summaries
- Structured patient-provided health data (e.g., age, BMI, lifestyle indicators)
- Optional medical image inputs (e.g., chest X-rays)

A number of fundamental elements were intended to make up the system architecture. Initially, a document-processing module would use OCR and PDF parsing to extract text from scanned and text-based PDF reports. Second, clinically relevant indicators like blood pressure readings, cholesterol levels, glucose levels, and other laboratory measurements would be found using rule-based Natural Language Processing (NLP) techniques. After that, these extracted values would undergo normalization, cleaning, and transformation into structured numerical features.

The third part used machine learning models that were taught by people like Logistic Regression and Random Forest classifiers to look at sets of features and create indicators that show the risk of getting a disease. The plan said it was very important that the models could be understood so it used ideas from AI to make sure the information shown to patients was clear. The system was not meant to replace what doctors think; it was meant to be a tool that helps doctors make decisions about patient care, specifically using machine learning models, like Logistic Regression and Random Forest classifiers, to support them.

Additionally, the proposal outlined the development of a patient-facing web interface that would:

- Allow secure upload of medical reports
- Display extracted and structured indicators
- Present risk outputs in a simplified, interpretable format

- Include ethical disclaimers emphasizing the non-diagnostic nature of the system

The research plan used numbers and experiments to get results. It included taking out information, teaching the model, checking how well it works, and comparing different algorithms. The team made sure to think about ethics, like using data to make it easy to understand and being responsible with artificial intelligence.

The goal of this research was to make a working model that shows how medical data from sources can be organized and analyzed to help patients know about possible health problems. The plan was to combine processing documents making features and machine learning that explains itself. This would help with research in healthcare and, with making digital health tools that focus on the patient. The research would help people in two ways: it would add to what we know about healthcare analytics. It would help make practical digital health applications that put the patient first.

3. Changes to the Proposal

While the core objective of the project remains aligned with the originally approved proposal, several refinements were made during implementation to ensure feasibility and technical stability.

3.1 Prioritization of Document Processing

The initial proposal was to have a system that could handle all kinds of information, like reports in PDF format, organized data, and medical pictures if needed. As we started working on it, we found out that getting information from PDF files and using special tools to read them is really important for everything else to work properly. The system really depends on being able to get information from PDF files and other documents in a way.

Therefore, the midterm focus was refined to prioritize:

- Secure PDF upload
- OCR-based text extraction
- Preprocessing and normalization
- Structured indicator generation
- Web-based presentation of extracted outputs

The optional image-based module has been deferred to a later milestone.

Justification: Accurate feature extraction is essential before implementing machine learning or image analysis. This phased approach strengthens system reliability.

3.2 Architectural Consolidation Using Streamlit

The proposal initially considered a separation between backend services and frontend components. During development, Streamlit was adopted as the primary integrated framework for both interface and processing logic.

Justification: Streamlit is really helpful for trying out new ideas and easily combining the parts that help us get the information we need and show it in a way that makes sense. This is what we need when we are working on a research project at school, and we have to get it done on time. Streamlit makes it easy to work with the components that help us extract information and visualize it, which is perfect for a research prototype that we are working on in school.

3.3 Phased Machine Learning Implementation

Although supervised models (e.g., Logistic Regression, Random Forest) were proposed , implementation of machine learning has been scheduled for the next phase following validation of structured feature extraction.

Justification: Model performance depends on clean and consistent feature engineering. Prioritizing data quality reflects best practices in applied machine learning workflows.

3.4 Refinement of Indicator Scope

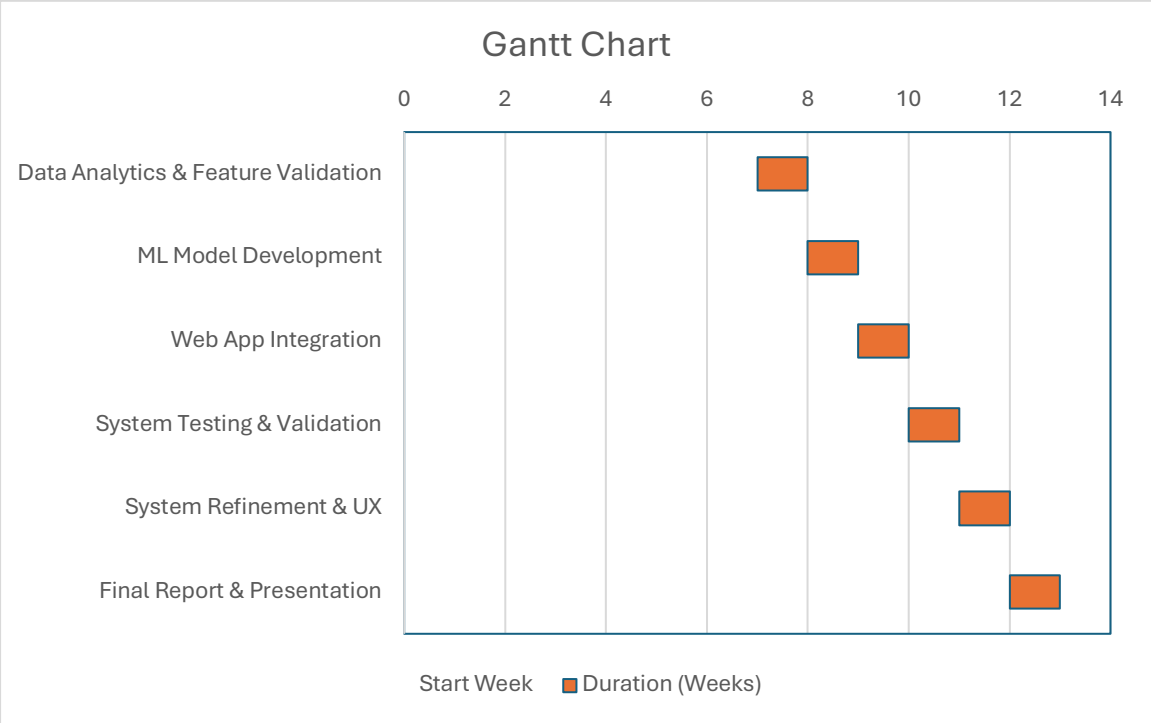
The proposal broadly referenced multiple disease categories. During implementation, the focus was refined to extracting generalized laboratory indicators (e.g., glucose, cholesterol, hematological values) that can later support multi-disease modeling.

This adjustment ensures alignment with the structure of patient-accessible reports while maintaining flexibility for future risk modeling.

4. Project Planning and Timeline (Midterm to End of Term)

Following completion of the midterm milestone, consisting of the integrated PDF/OCR extraction pipeline and structured indicator generation, the remaining phases focus on data analytics, machine learning experimentation, system integration, and validation. While the overall Gantt structure from the original proposal remains applicable, refinements have been made to emphasize data-driven analysis prior to full deployment.

4.1 Updated Timeline and Milestones



Weeks 7–8: Data Analytics and Feature Validation

Objectives:

- Perform exploratory data analysis (EDA) on extracted structured indicators
- Analyze distribution of laboratory values
- Validate normalization and consistency of extracted features
- Handle missing values and outliers
- Assess feature correlation and relevance for risk modeling

Deliverables:

- Cleaned and validated structured dataset
- Summary statistics and visualization outputs
- Feature selection justification report

Weeks 8–9: Machine Learning Model Development and Evaluation

Objectives:

- Implement supervised learning models (Logistic Regression, Random Forest)
- Split dataset into training and validation subsets
- Evaluate performance using accuracy, precision, recall, F1-score
- Compare interpretability and feature importance across models

Deliverables:

- Model performance comparison
- Feature importance analysis
- Selected model for deployment

Weeks 9–10: Integration of Analytics into Web Application

Objectives:

- Integrate selected model into Streamlit application
- Connect structured features to prediction pipeline
- Display risk level, confidence score, and contributing indicators
- Ensure outputs remain non-diagnostic and interpretable

Deliverables:

- Fully integrated analytics-enabled web application
- Updated repository with model pipeline

Weeks 10–11: System Testing and Data Validation

Objectives:

- Conduct end-to-end testing with multiple PDF inputs
- Validate robustness of extraction and prediction
- Evaluate performance under noisy or incomplete inputs
- Refine preprocessing and feature engineering rules

Deliverables:

- Stability testing documentation
- Improved error handling
- Finalized data processing workflow

Weeks 11–12: System Refinement and Interpretability Enhancement

Objectives:

- Improve clarity of analytic outputs
- Enhance visualization of risk indicators
- Simplify patient-facing explanations
- Validate ethical framing and transparency

Deliverables:

- Refined UI/UX
- Improved explanation of analytical results

Weeks 12–13: Final Report and Presentation

Objectives:

- Document full research methodology
- Present analytical findings and model comparisons
- Prepare defense presentation and live system demonstration

- Finalize GitHub documentation and installation instructions

Deliverables:

- Final research report
- Oral defense presentation
- Complete GitHub repository

4.2 Individual Responsibilities (From Midterm Onward)

Uthara Uday

- Lead exploratory data analysis and feature validation
- Implement and evaluate machine learning models
- Conduct comparative model performance analysis
- Document data analytics methodology and results

Akiful Mohaimin

- Maintain and refine PDF/OCR extraction pipeline
- Integrate analytical models into Streamlit application
- Perform multi-report validation testing
- Contribute to UI improvements and deployment readiness

Both team members will collaborate on:

- Data validation
- Ethical review and interpretability assessment
- Final documentation and presentation preparation

5. Implemented Features (Midterm Milestone)

The midterm stage involves the project being at the point where conceptual design is now referred to as a working prototype. The features that have been implemented revolve around PDF processing, structured data extraction, and a patient-facing web interface. The subsequent sections explain the parts of the system that were created within Weeks 4-7, as well as personal contributions.

5.1 Secure PDF Upload and Validation Module

Feature Description

A secure file upload mechanism was implemented within the Streamlit application to allow users to upload medical reports in PDF format. The system validates:

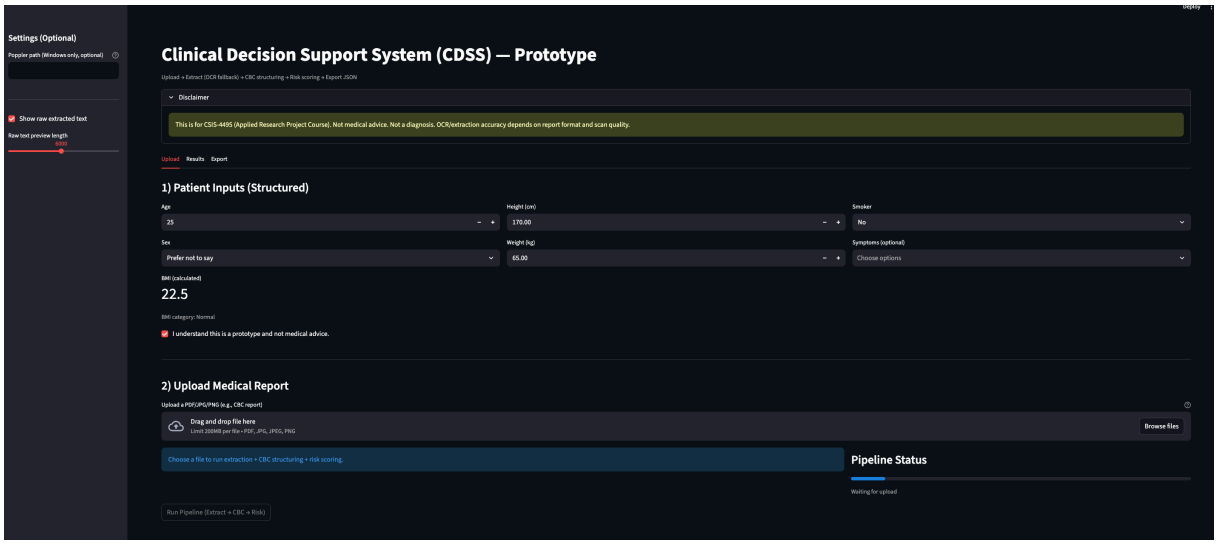
- File extension (.pdf, .jpg, .png where applicable)
- Maximum file size
- Existence and path security

Uploaded files are temporarily stored in a controlled directory to prevent unsafe file access.

Design Considerations

- Prevent arbitrary file execution
- Ensure compatibility with both text-based and scanned PDFs
- Provide immediate user feedback if file type is invalid

Screenshot of the interface in Streamlit



Screenshot of code:

```python
# ==========================================================
# Stage 1: Secure file checks
# ==========================================================
def validate_input_file(file_path: str) -> Path:
    """Basic safety checks: file exists, allowed extension, max size."""
    p = Path(file_path)

    if not p.exists():
        raise FileNotFoundError(f"File not found: {p.resolve()}")

    ext = p.suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise ValueError(f"Unsupported file type: {ext}. Allowed: {sorted(ALLOWED_EXTENSIONS)}")

    size_mb = p.stat().st_size / (1024 * 1024)
    if size_mb > MAX_FILE_MB:
        raise ValueError(f"File too large: {size_mb:.2f} MB (max {MAX_FILE_MB} MB)")

    return p
```

```python
# ------------------------
# TAB 1: Upload
# ------------------------

with tab_upload:
    st.subheader("1) Patient Inputs (Structured)")

    pi = st.session_state.patient_inputs

    c1, c2, c3 = st.columns(3)
    with c1:
        pi["age"] = st.number_input("Age", min_value=0, max_value=120, value=int(pi["age"]))
        pi["sex"] = st.selectbox(
            "Sex",
            ["Prefer not to say", "Female", "Male", "Other"],
            index=["Prefer not to say", "Female", "Male", "Other"].index(pi["sex"]),
        )

    with c2:
        pi["height_cm"] = st.number_input("Height (cm)", min_value=50.0, max_value=250.0, value=float(pi["height_cm"]), step=0.5)
        pi["weight_kg"] = st.number_input("Weight (kg)", min_value=10.0, max_value=300.0, value=float(pi["weight_kg"]), step=0.5)

    with c3:
        pi["smoker"] = st.selectbox("Smoker", ["No", "Yes"], index=["No", "Yes"].index(pi["smoker"]))
        pi["symptoms"] = st.multiselect(
            "Symptoms (optional)",
            ["Fatigue", "Fever", "Cough", "Shortness of breath", "Dizziness", "Chest pain", "Other"],
            default=pi["symptoms"],
        )

    bmi_value, bmi_category = compute_bmi(pi["height_cm"], pi["weight_kg"])
    st.metric("BMI (calculated)", "—" if bmi_value is None else str(bmi_value))
    st.caption(f"BMI category: {bmi_category}")
```

Individual Contribution

Akiful Mohaimin:

- Implemented secure upload logic
- Managed file handling and size control
- Resolved environment and path configuration issues

5.2 PDF Parsing and OCR Extraction Pipeline

Feature Description

To handle heterogeneous report formats, a dual extraction pipeline was implemented:

1. **Text-based PDFs:** Parsed using PDF libraries to extract embedded text.
2. **Scanned PDFs:** Processed using Tesseract OCR to convert image-based text into machine-readable format.

The system automatically attempts embedded text extraction and falls back to OCR when necessary.

Design Considerations

- Support real-world patient-accessible documents
- Handle noisy text extraction
- Maintain processing efficiency

Implementation Details

- Text extraction module
- OCR fallback mechanism
- Integration with the preprocessing stage

Screenshot of Extracted text preview in UI

CBC Indicators

	Indicator	Value	Unit	Reference	Flag
0	Basophils %	1.0000		<2	Normal
1	Eosinophils %	1.0000		1-6	Normal
2	HCT %	42.0000	%	40 - 50	Normal
3	Hemoglobin	15.0000		13-17	Normal
4	MCH	30.0000		27 - 32	Normal
5	MCV	84.0000		83 - 101	Normal
6	Neutrophils %	79.0000		40 - 80	Normal
7	Platelet Count	3.5000		lakhs/cumm	Unknown
8	Total RBC Count	5.0000		million/cumm	Unknown

Screenshot of code:

cbc_pipeline.py > ...

```python
# Stage 1: Main extraction pipeline
# ========================================================


def extract_report_text(file_path: str, poppler_path: Optional[str] = None) -> Dict[str, Any]:
    """Returns a record dict with patient ID and extracted raw text."""
    p = validate_input_file(file_path)

    record = {
        "patient_id": f"AUTO_GEN_{uuid.uuid4().hex[:8].upper()}",
        "source_file": p.name,
        "raw_text": "",
        "processed_indicators": {},
        "status": "Incomplete",
    }

    try:
        if p.suffix.lower() == ".pdf":
            text = extract_embedded_pdf_text(p)
            if len(text.strip()) < 50:
                text = ocr_pdf_to_text(p, poppler_path=poppler_path)
        else:
            text = ocr_image_to_text(p)

        record["raw_text"] = text
        record["status"] = "Extracted"
        return record

    except Exception as e:
        record["status"] = "Failed"
        record["error"] = repr(e)
        return record
```

Individual Contribution

Akiful Mohaimin:

- Designed and implemented PDF + OCR extraction workflow
- Debugged path and dependency configuration
- Integrated preprocessing logic into extraction output

5.3 Preprocessing and Structured Indicator Extraction

Feature Description

Raw OCR output typically contains irrelevant text, headers, footers, and formatting artifacts. A preprocessing module was developed to:

- Remove noisy characters
- Normalize whitespace
- Filter clinically relevant lines
- Extract numeric laboratory values using rule-based logic (e.g., regular expressions)

Extracted indicators are structured into key-value format for downstream analytics.

Design Considerations

- Preserve clinically relevant information
- Maintain interpretability
- Ensure consistency for future machine learning input

Implementation Details

- Regex-based extraction rules
- Schema definition for structured indicators
- Normalization of lab values

Screenshot of JSON output

Deploy

JSON Preview


```
{
  "patient_id": "AUTO_GEN_41941A89",
  "source_file": "uploaded.pdf",
  "raw_text": "~ Page 1 -\nRegd. No.: XXXX54826XX\nLabsmart Software beh a hla\n@ yourlabname@gmail.com\nScan to download\nMr. Saubhik Bhaumik EULA 00: OP eC)
  "processed_indicators": {
    "cbc": {
      "Hemoglobin": {
        "value": 15.0,
        "unit": null,
        "ref_raw": "13-17",
        "ref_low": 13.0,
        "ref_high": 17.0,
        "flag": "Normal"
      },
      "Neutrophils %": {
        "value": 79.0,
        "unit": null,
        "ref_raw": "40 - 80",
        "ref_low": 40.0,
        "ref_high": 80.0,
        "flag": "Normal"
      },
      "Eosinophils %": {
        "value": 1.0,
        "unit": null,
        "ref_raw": "1-6",
        "ref_low": 1.0,
        "ref_high": 6.0,
        "flag": "Normal"
      },
      "Basophils %": {
        "value": 1.0,
        "unit": null,
        "ref_raw": "<2",
```


Screenshot of code:

```python
CBC_PATTERNS = {
    "Hemoglobin": r"HEMOGLOBIN\s+(\d+(\.\d+)?)\s*(g\/d[lL])?\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Total Leukocyte Count": r"TOTAL\s+LEU[CK]OCYTE\s+COUNT\s+(\d+)\s*(cumm|\/cumm)?\s+(\d+\s*[-–]\s*\d+)",
    "Neutrophils %": r"NEUTROPHILS\s+(\d+(\.\d+)?)\s*%\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Lymphocyte %": r"LYMPHOCY?TE\s+(\d+(\.\d+)?)\s*%\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Eosinophils %": r"EOSINOPHILS\s+(\d+(\.\d+)?)\s*%\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Monocytes %": r"MONOCYTES\s+(\d+(\.\d+)?)\s*%\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Basophils %": r"BASOPHILS\s+(\d+(\.\d+)?)\s*%\s+(<\s*\d+(\.\d+)?|\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Platelet Count": r"PLATELET\s+COUNT\s+(\d+(\.\d+)?)\s*(lakhs\/cumm)?\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "Total RBC Count": r"TOTAL\s+RBC\s+COUNT\s+(\d+(\.\d+)?)\s*(million\/cumm)?\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "HCT %": r"(HCT|HEMATOCRIT\s+VALUE,\s*HCT)\s+(\d+(\.\d+)?)\s*%\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "MCV": r"\bMCV\b\s+(\d+(\.\d+)?)\s*fL\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "MCH": r"\bMCH\b\s+(\d+(\.\d+)?)\s*pg\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
    "MCHC": r"\bMCHC\b\s+(\d+(\.\d+)?)\s*%\s+(\d+(\.\d+)?\s*[-–]\s*\d+(\.\d+)?)",
}


def extract_cbc(text: str) -> Dict[str, Dict[str, Any]]:
    """Extract CBC fields from raw text using regex."""
    results = {}
    for indicator, pattern in CBC_PATTERNS.items():
        match = re.search(pattern, text, flags=re.IGNORECASE)
        if not match:
            continue
        if indicator == "HCT %":
            value = safe_float(match.group(2))
            ref_raw = match.group(4)
            unit = "%"
        else:
            value = safe_float(match.group(1))
            ref_raw = match.group(4) if indicator == "Hemoglobin" else match.group(3)
            unit = None
        low, high = parse_reference_range(ref_raw)
        results[indicator] = {
            "value": value,
            "unit": unit,
            "ref_raw": ref_raw,
            "ref_low": low,
            "ref_high": high,
            "flag": classify_flag(value, low, high),
        }
    return results
```

Individual Contribution

Uthara Uday:

- Designed preprocessing strategy
- Defined schema for structured indicators
- Implemented normalization and feature structuring logic
- Validated indicator extraction accuracy

5.4 Integrated Streamlit Web Application

Feature Description

The Streamlit application (sampleapp.py) serves as the patient-facing interface and integrates:

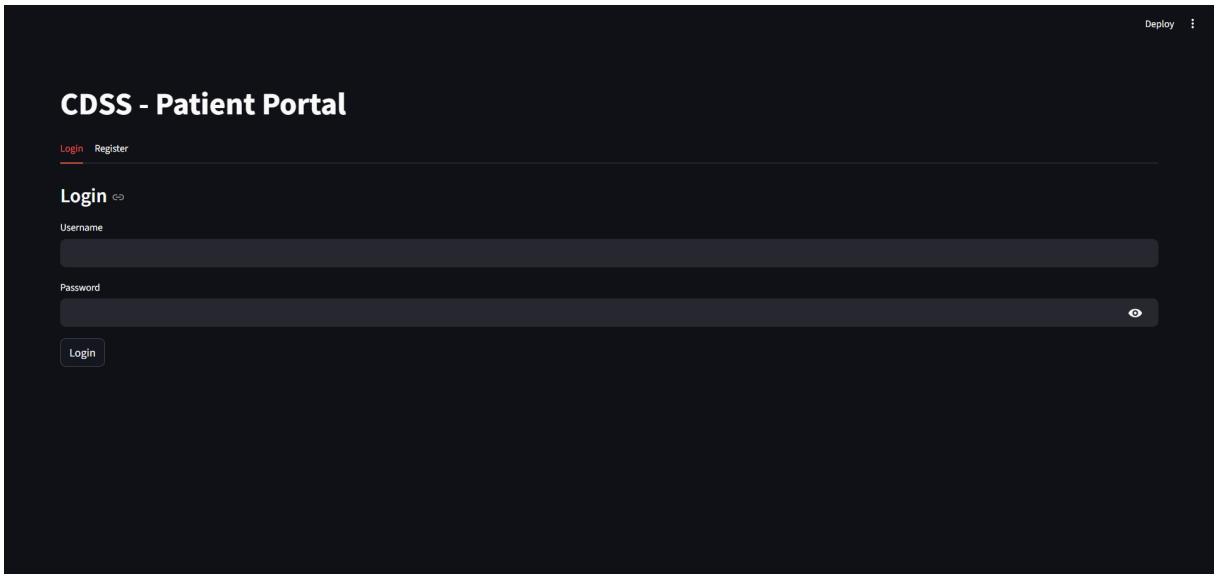
- File upload
- Extraction pipeline
- Preprocessing module
- Structured output display
- Ethical non-diagnostic disclaimer

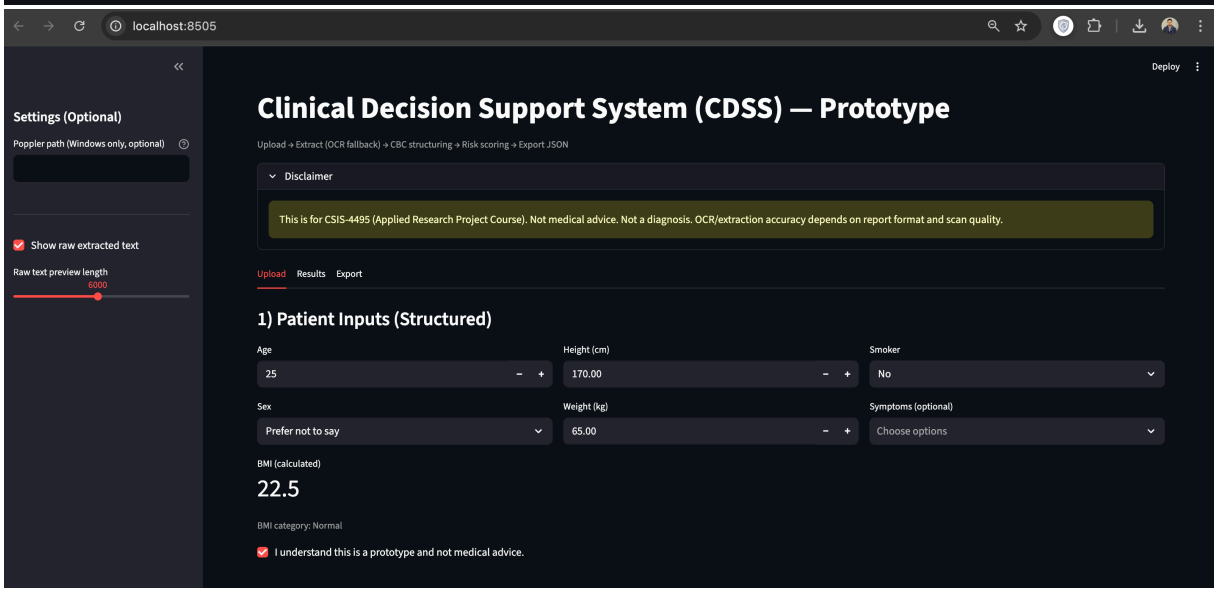
The application allows users to upload a medical report and view structured extracted indicators in a readable format.

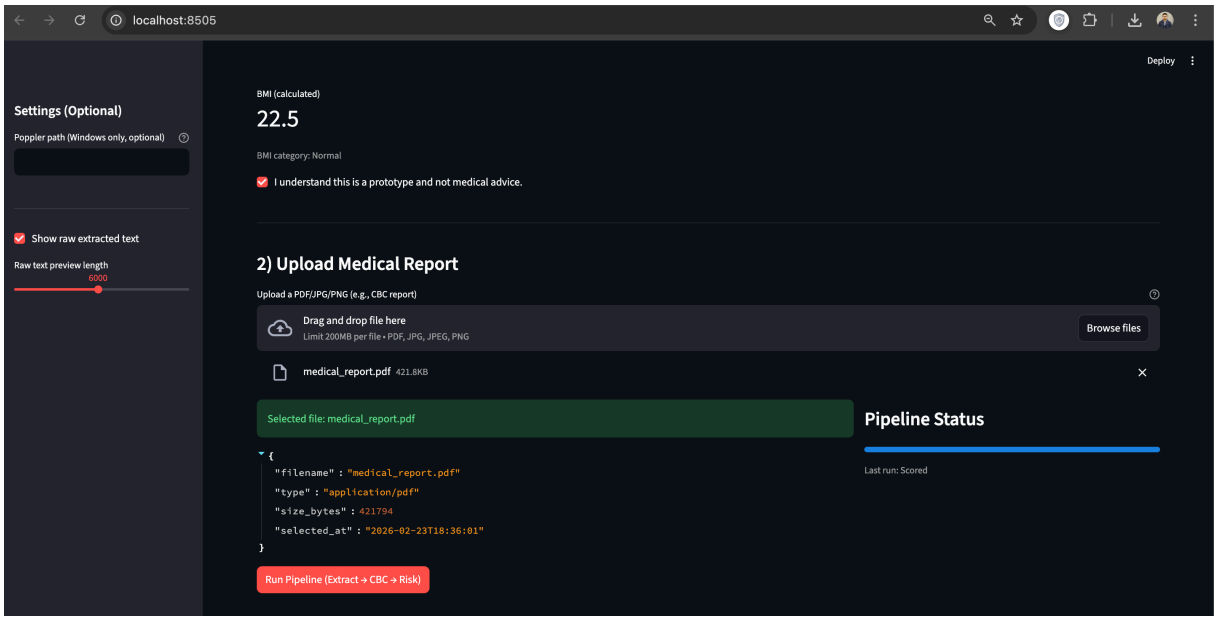
Design Considerations

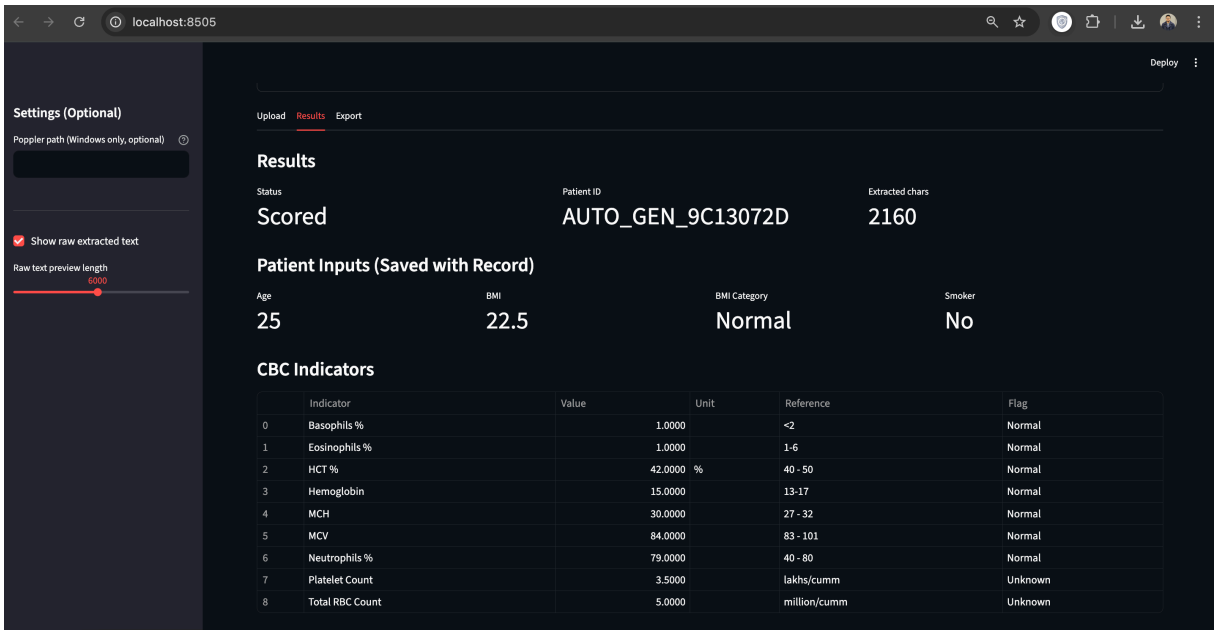
- Simplicity and clarity for non-clinical users
- Transparent presentation of extracted values
- Separation between informational support and diagnosis

Screenshot of the full application interface:

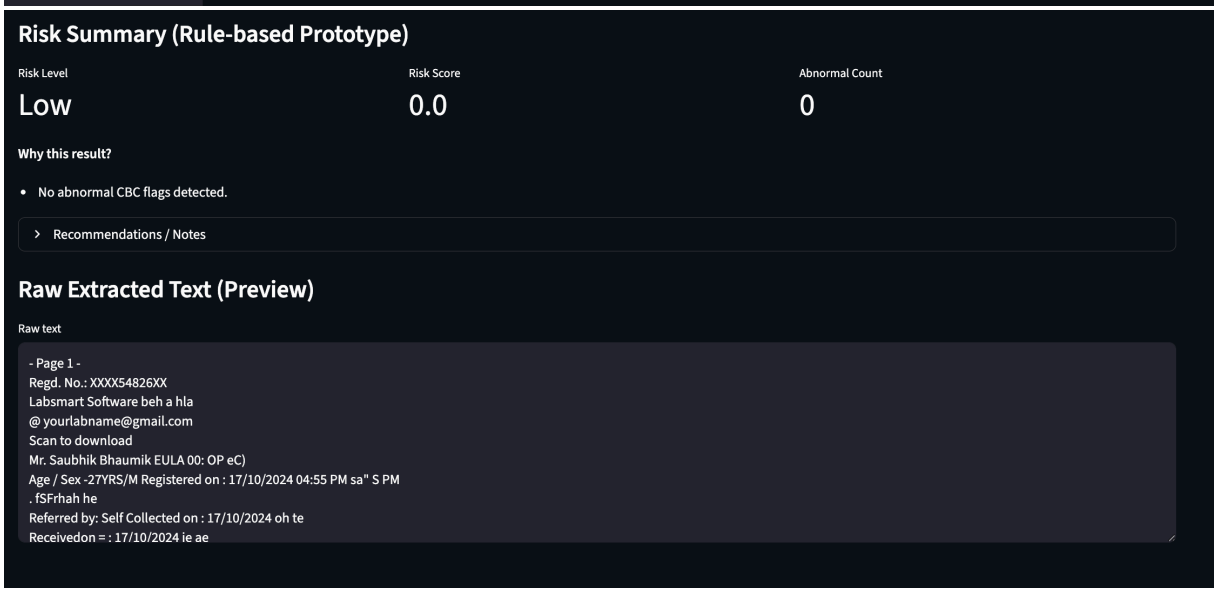








	Indicator	Value	Unit	Reference	Flag
0	Basophils %	1.0000		<2	Normal
1	Eosinophils %	1.0000		1-6	Normal
2	HCT %	42.0000	%	40 - 50	Normal
3	Hemoglobin	15.0000		13-17	Normal
4	MCH	30.0000		27 - 32	Normal
5	MCV	84.0000		83 - 101	Normal
6	Neutrophils %	79.0000		40 - 80	Normal
7	Platelet Count	3.5000		lakhs/cumm	Unknown
8	Total RBC Count	5.0000		million/cumm	Unknown



Screenshot of code:

```python
import json
import tempfile
from datetime import datetime
from pathlib import Path
from typing import Optional, Tuple

import streamlit as st

from cbc_pipeline import run_extraction_pipeline, extract_cbc_indicators
from risk_engine import compute_risk_from_cbc


# -------------------------
# Helpers
# -------------------------
def compute_bmi(height_cm: float, weight_kg: float) -> Tuple[Optional[float], str]:
    """Returns (bmi_value, bmi_category)."""
    if height_cm <= 0 or weight_kg <= 0:
        return None, "Invalid"
    height_m = height_cm / 100.0
    bmi = weight_kg / (height_m ** 2)

    # WHO categories (simple)
    if bmi < 18.5:
        cat = "Underweight"
    elif bmi < 25:
        cat = "Normal"
    elif bmi < 30:
        cat = "Overweight"
    else:
        cat = "Obese"
    return round(bmi, 1), cat


# -------------------------
# Page setup
# -------------------------

st.set_page_config(page_title="CDSS Prototype (Stage 1+2+3)", layout="wide")

st.title("Clinical Decision Support System (CDSS) — Prototype")
st.caption("Upload → Extract (OCR fallback) → CBC structuring → Risk scoring → Export JSON")

with st.expander("Disclaimer", expanded=True):
    st.warning(
        "This is for CSIS-4495 (Applied Research Project Course). Not medical advice. Not a diagnosis. "
        "OCR/extraction accuracy depends on report format and scan quality."
    )
```

sampleapp.py | cbc_pipeline.py | risk_engine.py

sampleapp.py > ...

```python
st.sidebar.header("Settings (Optional)")
poppler_path = st.sidebar.text_input(
    "Poppler path (Windows only, optional)",
    value="",
    help="If pdf2image fails on Windows, install Poppler and paste its /bin path here.",
).strip() or None

st.sidebar.markdown("---")
show_raw_text = st.sidebar.checkbox("Show raw extracted text", value=True)
raw_preview_chars = st.sidebar.slider("Raw text preview length", 500, 12000, 6000, step=500)

# Tabs
tab_upload, tab_results, tab_export = st.tabs(["Upload", "Results", "Export"])

# Shared state
if "record" not in st.session_state:
    st.session_state.record = None
if "cbc" not in st.session_state:
    st.session_state.cbc = None
if "risk" not in st.session_state:
    st.session_state.risk = None
if "patient_inputs" not in st.session_state:
    st.session_state.patient_inputs = {
        "age": 25,
        "sex": "Prefer not to say",
        "height_cm": 170.0,
        "weight_kg": 65.0,
        "smoker": "No",
        "symptoms": [],
        "consent": True,
    }

# -------------------------
# TAB 1: Upload
# -------------------------

with tab_upload:
    st.subheader("1) Patient Inputs (Structured)")

    pi = st.session_state.patient_inputs

    c1, c2, c3 = st.columns(3)
    with c1:
        pi["age"] = st.number_input("Age", min_value=0, max_value=120, value=int(pi["age"]))
        pi["sex"] = st.selectbox(
            "Sex",
            ["Prefer not to say", "Female", "Male", "Other"],
            index=["Prefer not to say", "Female", "Male", "Other"].index(pi["sex"]),
        )
```

```python
    with c2:
        pi["height_cm"] = st.number_input("Height (cm)", min_value=50.0, max_value=250.0, value=float(pi["height_cm"]), step=0.5)
        pi["weight_kg"] = st.number_input("Weight (kg)", min_value=10.0, max_value=300.0, value=float(pi["weight_kg"]), step=0.5)

    with c3:
        pi["smoker"] = st.selectbox("Smoker", ["No", "Yes"], index=["No", "Yes"].index(pi["smoker"]))
        pi["symptoms"] = st.multiselect(
            "Symptoms (optional)",
            ["Fatigue", "Fever", "Cough", "Shortness of breath", "Dizziness", "Chest pain", "Other"],
            default=pi["symptoms"],
        )

    bmi_value, bmi_category = compute_bmi(pi["height_cm"], pi["weight_kg"])
    st.metric("BMI (calculated)", "—" if bmi_value is None else str(bmi_value))
    st.caption(f"BMI category: {bmi_category}")

    pi["consent"] = st.checkbox("I understand this is a prototype and not medical advice.", value=bool(pi["consent"]))

    st.markdown("---")
    st.subheader("2) Upload Medical Report")

    uploaded = st.file_uploader(
        "Upload a PDF/JPG/PNG (e.g., CBC report)",
        type=["pdf", "jpg", "jpeg", "png"],
        help="File will be processed locally by the pipeline.",
    )

    col1, col2 = st.columns([2, 1])

    with col1:
        if uploaded:
            st.success(f"Selected file: {uploaded.name}")
            st.json(
                {
                    "filename": uploaded.name,
                    "type": uploaded.type,
                    "size_bytes": uploaded.size,
                    "selected_at": datetime.now().isoformat(
timespec="seconds"),

                }
            )
        else:
            st.info("Choose a file to run extraction + CBC structuring + risk scoring.")

    with col2:
        st.markdown("### Pipeline Status")
        rec = st.session_state.record
        if rec is None:
```

```python
# -------------------------
# TAB 2: Results
# -------------------------

with tab_results:
    st.subheader("Results")

    rec = st.session_state.record
    cbc = st.session_state.cbc
    risk = st.session_state.risk

    if rec is None:
        st.info("No run yet. Go to Upload tab → fill inputs → upload a file → click Run.")
    else:
        m1, m2, m3 = st.columns(3)
        m1.metric("Status", rec.get("status", ""))
        m2.metric("Patient ID", rec.get("patient_id", ""))
        m3.metric("Extracted chars", str(len(rec.get("raw_text", ""))))

        st.markdown("### Patient Inputs (Saved with Record)")
        pi = rec.get("patient_inputs", {})
        p1, p2, p3, p4 = st.columns(4)
        p1.metric("Age", str(pi.get("age", "")))
        p2.metric("BMI", "–" if pi.get("bmi") is None else str(pi.get("bmi")))
        p3.metric("BMI Category", str(pi.get("bmi_category", "")))
        p4.metric("Smoker", str(pi.get("smoker", "")))


        st.markdown("### CBC Indicators")
        if not cbc:
            st.warning("No CBC fields detected. (This can happen if OCR/text format differs.)")
        else:
            rows = []
            for name, v in cbc.items():
                rows.append(
                    {
                        "Indicator": name,
                        "Value": v.get("value"),
                        "Unit": v.get("unit") or "",
                        "Reference": v.get("ref_raw") or "",
                        "Flag": v.get("flag") or "",
                    }
                )
            st.table(sorted(rows, key=lambda r: r["Indicator"]))

        st.markdown("### Risk Summary (Rule-based Prototype)")
        if not risk:
            st.info("Risk score not available yet.")
        else:
```

```python
                    "Flag": v.get("flag") or "",
                }
            )
        st.table(sorted(rows, key=lambda r: r["Indicator"]))

    st.markdown("### Risk Summary (Rule-based Prototype)")
    if not risk:
        st.info("Risk score not available yet.")
    else:
        r1, r2, r3 = st.columns(3)
        r1.metric("Risk Level", risk.get("risk_level", ""))
        r2.metric("Risk Score", str(risk.get("risk_score", "")))
        r3.metric("Abnormal Count", str(risk.get("abnormal_count", "")))

        st.markdown("**Why this result?**")
        for reason in risk.get("reasons", []):
            st.write(f"- {reason}")

        with st.expander("Recommendations / Notes"):
            for tip in risk.get("recommendations", []):
                st.write(f"- {tip}")

    if show_raw_text:
        st.markdown("### Raw Extracted Text (Preview)")
        raw = rec.get("raw_text", "")
        st.text_area("Raw text", raw[:raw_preview_chars], height=260)

# -------------------------
# TAB 3: Export
# -------------------------
with tab_export:
    st.subheader("Export JSON")

    rec = st.session_state.record
    if rec is None:
        st.info("Nothing to export yet. Run the pipeline first.")
    else:
        st.caption("This JSON is useful evidence for your progress report.")
        json_bytes = json.dumps(rec, indent=2).encode("utf-8")

        st.download_button(
            "Download structured JSON",
            data=json_bytes,
            file_name=f"{rec.get('patient_id','record')}_record.json",
            mime="application/json",
        )

        st.markdown("### JSON Preview")
        st.code(json.dumps(rec, indent=2), language="json")
```

Individual Contribution

Uthara Uday:

- Designed and implemented Streamlit interface
- Structured layout and input components
- Integrated extraction outputs into UI display

Akiful Mohaimin:

- Assisted with integration testing
- Ensured pipeline compatibility with UI
- Validated end-to-end workflow

5.5 End-to-End Workflow Validation

At the midterm stage, the system demonstrates the following end-to-end workflow:

1. User uploads PDF medical report
2. System performs text extraction (PDF or OCR)
3. Preprocessing filters noisy text

4. Structured indicators are generated
5. Results are displayed in a patient-readable format

This milestone establishes the foundational data-processing pipeline required for subsequent data analytics and machine learning integration.

6. AI Use Section

AI Tool Name	Version, Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition (What We Did Beyond AI)
ChatGPT	GPT-5.2, Premium	Identification of commonly reported clinical indicators (heart disease, diabetes, stroke, breast cancer)	Completed our own research on indicators for future modifications in the project
ChatGPT	GPT-5.2, Premium	Suggestions for Streamlit UI structuring before backend integration	Independently designed UI layout, implemented upload interface, integrated extraction outputs, and refined disclaimer placement
ChatGPT	GPT-5.2, Premium	Selection of suitable machine learning algorithms for healthcare risk prediction	Evaluated feasibility, planned comparative modeling strategy, and aligned algorithms with explainable AI principles
ChatGPT	GPT-5.2, Premium	Assistance in creating a gantt chart	Followed the steps and created the gantt chart in excel

7. Work Date / Hours Log

Student Name: Uthara Uday

Date	Hours	Description of Work Done
Feb 9, 2026	1.5	Refined preprocessing logic and discussed extraction inconsistencies with Akiful
Feb 10, 2026	2.0	Implemented structured indicator schema and coordinated integration points with extraction module
Feb 11, 2026	1.5	Joint testing session with Akiful to validate normalization of extracted values
Feb 13, 2026	2.0	Integrated structured output into Streamlit interface and reviewed end-to-end workflow
Feb 16, 2026	2.0	Improved UI formatting based on joint feedback session
Feb 18, 2026	1.5	Discussed feature engineering approach for upcoming ML phase with team member

Feb 20,2026	2	Added a login and registration module
Feb 22, 2026	2.0	Reviewed technical details and documentation with Akiful
Feb 23, 2026	1.5	Final integration check and rehearsal of midterm demo together

Student Name: Akiful Mohaimin

Date	**Hours**	**Description of Work Done**
Feb 9, 2026	1.5	Fixed OCR fallback mechanism and reviewed preprocessing compatibility with Uthara
Feb 11, 2026	2.0	Tried to optimize file validation and integration boundaries via Google meeting with Uthara
Feb 12, 2026	1.5	Cleaned & fixed the errors of the code of the extraction pipeline and shared outputs
Feb 14, 2026	2.0	Conducted testing of the built prototype
Feb 16, 2026	1.5	Tried to improve error handling mechanism and verified structured outputs with UI component
Feb 18, 2026	2.0	Integrated preprocessing module outputs and confirmed consistency through joint review and discussed approaches for the ML phase with teammate
Feb 20, 2026	1.5	Verified compatibility between extraction pipeline and Streamlit display logic
Feb 22, 2026	1.5	Organized GitHub repository, reviewed folder structure with Uthara and also reviewed the documentation
Feb 23, 2026	1.5	Participated in final validation of the prototype and midterm demo preparation

8. Closing

This midterm implementation goal is evidence of the effective transition from the conceptual design to the working prototype that is able to receive unstructured medical reports and provide meaningful indicators. Dynamic features put in place create a stable technical base upon which the subsequent data analytics and machine learning processes will occur.

The project still supports its initial goal of creating a patient-centered, non-diagnostic clinical decision support framework based on the ethical and explainable AI concepts.

8.1 References

1. Shortliffe, E. H., & Sepúlveda, M. J. (2018). Clinical decision support in the era of artificial intelligence. *JAMA, 320*(21), 2199–2200. https://doi.org/10.1001/jama.2018.17163
2. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine, 380*(14), 1347–1358. https://doi.org/10.1056/NEJMra1814259
3. Holzinger, A., Langs, G., Denk, H., Zatloukal, K., & Müller, H. (2019). Explainable artificial intelligence for healthcare. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 9*(4), e1312. https://doi.org/10.1002/widm.1312
4. Agency for Healthcare Research and Quality. (2025). *Clinical decision support systems (CDSS) primer*. PSNet. U.S. Department of Health & Human Services. https://psnet.ahrq.gov/primer/clinical-decision-support-systems
5. Agency for Healthcare Research and Quality. (n.d.). *Clinical decision support (CDS)*. U.S. Department of Health & Human Services. https://www.ahrq.gov/cpi/about/otherwebsites/clinical-decision-support/index.html
6. Wolters Kluwer. (2026). Can you trust your clinical decision support tool? https://www.wolterskluwer.com/en/expert-insights/can-you-trust-your-clinicaldecision-support-tool

9. APPENDIX

Appendix A – Prompt History

Prompt 1: What clinical indicators are commonly present in patient-accessible reports for assessing disease risk?

Prompt 2: What machine learning models are suitable for structured healthcare data and remain interpretable?

Prompt 3: How should I structure a Streamlit interface before connecting it to backend extraction logic?”

Prompt 4: How to create a gantt chart in excel?